



Matière : TP Système d'exploitation 2

TP : Gestion des processus

❖ Rappel

- Pour exécuter le programme en langage C sous Linux, suivez ces étapes :
 - Écrire le code source : Utilisez un éditeur de texte pour écrire le code source en langage C. Par exemple, vous pouvez utiliser nano, vi, vim, ou un éditeur de texte graphique.
 - Enregistrer le fichier : Enregistrez le fichier avec l'extension ".c" (par exemple, programme.c)
 - Compiler le programme : **gcc -o programme programme.c**
 - Exécuter le programme : **./programme**
- Pour installer un package sous Linux, utilisez :

sudo apt-get install nom_de_package

❖ Travail demandé

Exercice 1 :

1. Quelle est la fonction principale du programme ci-dessous? Expliquez le rôle de la fonction fork et sleep dans ce programme.
2. Lancer le programme ci-dessous avec les arguments **10 20**. Taper **ps -la** dans un autre terminal avant la fin du père, avant la fin du fils. Quels sont les PPID du père et du fils ? Donner une explication.
3. Lancer le programme ci-dessous avec les arguments **10 0**. Tapez **ps -la** dans un autre terminal avant la fin du père. Que constatez-vous ?
4. Considérez la situation où le temps d'attente du fils (attente_fils) est plus long que le temps d'attente du père (attente_pere). Supposons que 'attente_pere' soit égal à 5 secondes et 'attente_fils' soit égal à 10 secondes. Comment cela affecterait-il l'ordre d'affichage des messages "père attente finie" et "fils attente finie"? Expliquez.
5. Modifiez le code pour afficher le PID, le propriétaire et le groupe du processus père et du processus fils. Comment cette modification affecterait-elle la sortie du programme?

exercice1.c

```

#include <unistd.h> /* necessaire pour les fonctions exec */
#include <sys/types.h> /* necessaire pour la fonction fork */
#include <unistd.h> /* necessaire pour la fonction fork */
#include <stdio.h> /* necessaire pour la fonction perror */

int main(int argc, char * argv[]) {
    pid_t pid;
    int attente_fils, attente_pere;

    if(argc != 3)
    {
        printf("usage: ex1 'attente_pere' 'attente_fils'\n");
        exit(-1);
    }

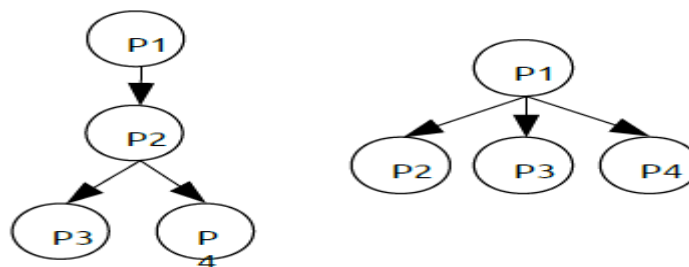
    attente_pere = atoi(argv[1]);
    attente_fils = atoi(argv[2]);

    switch(pid=fork()) {
        case -1:
            perror("fork error");
            break;
        case 0:
            sleep(attente_fils);
            printf("fils attente finie\n");
            break;
        default:
            sleep(attente_pere);
            printf("pere attente finie\n");
            break;
    }
    return 0;
}

```

Exercice 2:

1. A l'aide du man de la commande ps trouver une commande permettant d'afficher l'arbre des processus issue d'un processus initial.
2. Écrire un programme qui permet de créer chacune des arborescences de processus ci-dessous. Chaque processus doit afficher son nom (indiqué dans la figure), son identifiant et l'identifiant de son père.



3. Écrire un programme qui affiche l'arbre des processus du processus courant (utiliser la primitive **system**).

Exercice 3 :

1. Écrire un programme qui permet de créer un processus P1 qui a un processus fils P2.
2. Ajouter les instructions qui permettront au processus père d'attendre la terminaison de son fils et qui afficheront le code retour de celui-ci. On suppose dans un premier temps que le fils se termine correctement : `exit(0)`.
3. Ajouter les instructions qui permettront au processus père d'interpréter les informations de terminaison de son processus fils :

- Code retour du processus fils
- Signal ayant provoqué la terminaison du processus fils.

On fera terminer le processus fils avec un code retour de 3.

4. Mettre ensuite le processus fils en sommeil (`sleep (10)`) et tuer ensuite celui-ci à l'aide de la commande shell : `kill -15 pid_fils`.

On prendra soin de lancer le programme en arrière plan. Interpréter le résultat obtenu.

5. Reproduire le cas précédent avec « `kill -9 pid_fils` ».

Exercice 4:

1. Rappeler c'est quoi un processus zombie.
2. Écrire un programme simple qui permet de créer un processus zombie.
3. Modifier ce programme pour éliminer le processus zombie.

Exercice 5:

Le programme suivant permet d'illustrer le comportement de 'exec'.

```
#include <stdio.h>
#include <unistd.h>

main () {
    char buf [80];

    printf ("%d)-- Debut du test exec () \n",getpid());
    sprintf (buf,"-- pid=[%d]", getpid());
    execl ("/bin/echo","echo","Execution","d'un test exec",buf, NULL);
    printf ("%d)-- Echec de la fonction execl () \n",getpid());
    printf ("%d)-- Fin du test exec () \n",getpid());
    exit (0);
}
```

1. Exécuter le code puis expliquer le résultat obtenu.
2. Remplacer la commande « `/bin/echo` » par « `/bin/ech` ». Expliquer le résultat obtenu.

Exercice 6:

1. A l'aide des primitives fork et execl, écrire un programme qui permet de faire exécuter la commande ps avec l'option -l depuis un processus existant. Ce dernier devra se mettre en attente du code retour de la commande ps.
2. Reprendre le programme précédent mais en utilisant la commande execv.

Exercice 7 :

Écrire un programme dans le quel le processus fils écrase le code hérité de son père par celui de la commande **ls**, pour exécuter la commande « **ls -l /** » qui liste l'ensemble des fichiers du répertoire racine.